

Multi-Container Todo Application Using Docker Networking and MySQL

1. Introduction

Modern applications are often built using a microservices or multi-container architecture. Instead of running all components inside a single container, each service runs independently in its own container. This project demonstrates how to run a **Node.js Todo application** and a **MySQL database** in separate Docker containers and connect them using **Docker container networking**.

2. Objective

- To understand the concept of multi-container applications
- To deploy MySQL as a separate container
- To connect an application container to a database container using Docker networking
- To use Docker volumes for persistent database storage
- To verify data persistence and inter-container communication

3. Prerequisites

- Ubuntu Operating System
- Docker installed and running
- Basic knowledge of Docker commands
- Internet connection to pull Docker images

4. Project Architecture

Components Used:

- Application Container: Node.js Todo App
- Database Container: MySQL 8.0
- Network: Docker bridge network (todo-app)
- Volume: todo-mysql-data

Architecture Flow:

User → Browser → Node.js Container → MySQL Container → Docker Volume

5. Step-by-Step Implementation

Step 1: Create a Docker Network

```
docker network create todo-app
```

Explanation:

- Creates a custom Docker bridge network
- Allows containers to communicate using container names
-

```
admin@RitejMachine:~$ docker network create todo-app
0701904b44fce1033a49b25cbad93d0bfd9354b7f9cda346248fe21835f8d21d
admin@RitejMachine:~$ |
```

Docker network creation

Step 2: Start MySQL Container on the Network

```
docker run -d \
--network todo-app --network-alias mysql \
-v todo-mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=todos \
mysql:8.0
```

Explanation:

- Runs MySQL in detached mode
- Attaches container to todo-app network
- Sets network alias as mysql
- Uses a named volume for persistent storage
- Initializes database with name todos

```
admin@RitejMachine:~$ docker run -d \
--network todo-app --network-alias mysql \
-v todo-mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=todos \
mysql:8.0
abeefc8bf732c6a3c45e07968356d665d30232763c0015c2e584714b05238122
admin@RitejMachine:~$
```

MySQL container running

Step 3: Verify MySQL Database

`docker exec -it <mysql-container-id> mysql -u root -p`

Enter password:

secret

`SHOW DATABASES;`

Expected Output:

todos

```
admin@RitejMachine:~$ docker exec -it abeefc8bf732 mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| todos |
+-----+
5 rows in set (0.03 sec)

mysql>
```

MySQL database verification

Exit MySQL shell:

`exit`

Step 4: Verify Container Networking Using Netshoot

`docker run -it --network todo-app nicolaka/netshoot`

Inside the container:

dig mysql

Explanation:

- Confirms Docker DNS resolves mysql to the MySQL container IP
- Demonstrates service discovery using Docker networking

```
admin@RitejMachine:~$ docker run -it --network todo-app nicolaka/netshoot
      dP      dP      dP
      88      88      88
88d888b. .d8888b. d8888P .d8888b. 88d888b. .d8888b. .d8888b. d8888P
88' `88 88oooo8 88 Y8oooo. 88' `88 88' `88 88' `88 88
88 88 88. ... 88      88 88 88 88. .88 88. .88 88
dP dP `88888P' dP `88888P' dP dP `88888P' `88888P' dP

Welcome to Netshoot! (github.com/nicolaka/netshoot)
Version: 0.14

58f634d61b5d > dig mysql

; <<>> DiG 9.20.10 <<>> mysql
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 56469
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;mysql.                IN      A

;; ANSWER SECTION:
mysql.                600     IN      A      172.18.0.2

;; Query time: 10 msec
;; SERVER: 127.0.0.11#53(127.0.0.11) (UDP)
;; WHEN: Sat Dec 13 16:52:20 UTC 2025
;; MSG SIZE rcvd: 44
```

DNS resolution of MySQL container

Exit netshoot container:

exit

Step 5: Run the Todo Application Container

Make sure you are inside the getting-started-app directory.

```
docker run -dp 127.0.0.1:3000:3000 \
-w /app -v "$(pwd)/app" \
--network todo-app \
-e MYSQL_HOST=mysql \
-e MYSQL_USER=root \
-e MYSQL_PASSWORD=secret \
-e MYSQL_DB=todos \
node:lts-alpine \
sh -c "yarn install && yarn run dev"
```

Explanation:

- Runs Node.js app container
- Mounts application source code
- Connects container to MySQL via Docker network
- Uses environment variables for database connection

```
admin@RitejMachine:~$ docker run -dp 127.0.0.1:3000:3000 \
-w /app -v "$(pwd):/app" \
--network todo-app \
-e MYSQL_HOST=mysql \
-e MYSQL_USER=root \
-e MYSQL_PASSWORD=secret \
-e MYSQL_DB=todos \
node:lts-alpine \
sh -c "yarn install && yarn run dev"
16d7e8f231c96f80b98090e144b4d62ff6bd0c1fa2dc28ff249b0ee921b5d1f5
admin@RitejMachine:~$ |
```

Application container logs

Step 6: Verify Application Logs

`docker logs -f <app-container-id>`

Expected Logs:

Connected to mysql db at host mysql
Listening on port 3000

```
admin@RitejMachine:~/getting-started-app$ docker logs -f 3debc735bb1d
Using sqlite database at /etc/todos/todo.db
Listening on port 3000
|
```

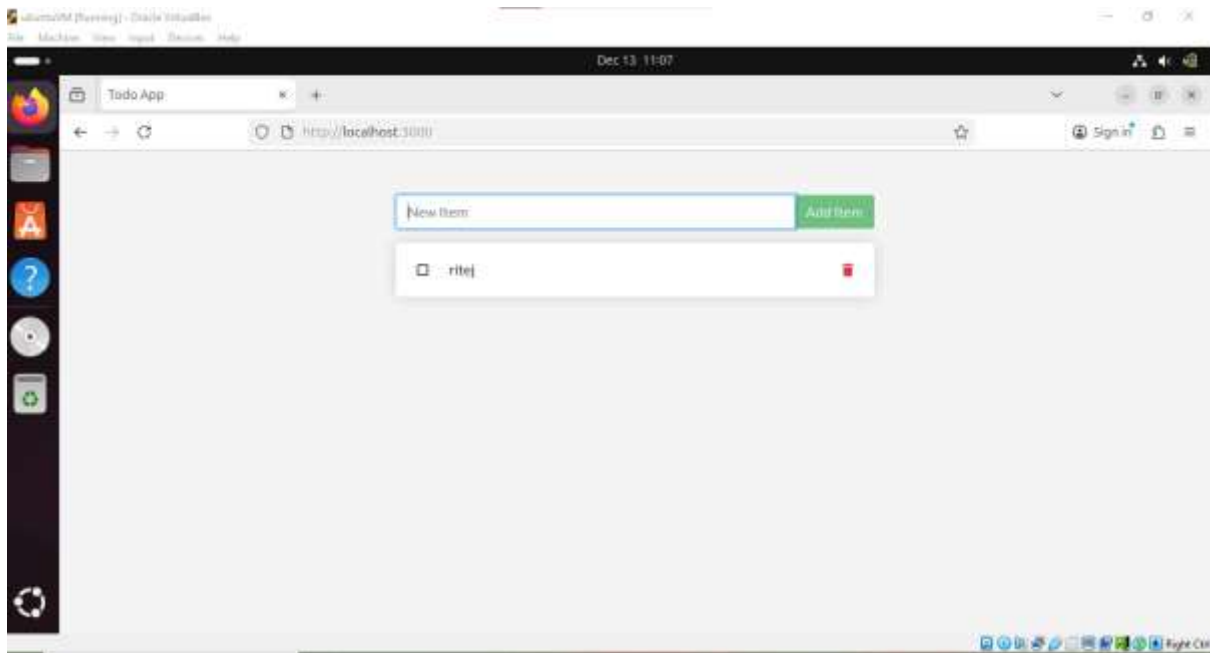
Successful database connection log

Step 7: Access the Application

Open browser and visit:

`http://localhost:3000`

Add a few todo items using the UI.



Todo app UI with added items

Step 8: Verify Data Stored in MySQL

`docker exec -it <mysql-container-id> mysql -u root -p todos`

Enter password:

secret

`SELECT * FROM todo_items;`

Expected Output:

Records inserted by the application

```
mysql> SELECT * FROM todo_items;
+----+-----+-----+
| id | name | completed |
+----+-----+-----+
| NULL | ritej | NULL |
+----+-----+-----+
1 row in set (0.01 sec)

mysql> |
```

Database records verification

6. Result

The Node.js Todo application successfully communicates with the MySQL database running in a separate container. Data entered through the application UI is persistently stored in the MySQL database using Docker volumes.

7. Conclusion

This project demonstrates a real-world **multi-container Docker application** using container networking, DNS-based service discovery, and persistent storage. Separating application and database containers improves scalability, maintainability, and production readiness.

8. Key Learnings

- Multi-container architecture principles
- Docker bridge networking
- Container-to-container communication using DNS
- MySQL data persistence using Docker volumes
- Environment-based configuration

9. Future Scope

- Convert setup to Docker Compose
- Use secrets instead of environment variables
- Deploy on cloud platforms
- Replace MySQL with managed database services